Implementation Note — M25DE1042 | Speech A2 Assignment

IIT Jodhpur · MTech Data Engineering · Speech Understanding Using AI

Q1 — Constrained ASR: Non-Obvious Design Choice in N-Gram Logit Biasing

Choice: Selective biasing restricted to `V_tech` token IDs, not the full vocabulary.

The naive approach would bias all vocabulary tokens using the trigram LM at every decoding step. With Whisper's vocabulary of ~50,000 BPE tokens, this would require 50,000 LM lookups per beam, per step, per beam-width — roughly 250,000 LM queries per token generation step. On CPU, this renders real-time decoding infeasible.

The non-obvious fix is to pre-compute `V_tech`: the set of token IDs corresponding to the 85 domain-specific words (e.g., *spectrogram*, *Wav2Vec2*, *MFCC*, *DTW*). Each word is encoded both with and without a leading space to handle subword tokenization (e.g., `" mfcc"` vs `"mfcc"`), yielding approximately 427 unique token IDs. At each decoding step, the inner loop iterates over only these 427 IDs rather than the full 50k vocabulary. This reduces the per-step overhead by **99.1%** while still providing meaningful guidance for domain terminology.

A second subtle point: biasing is applied **in log-space** (`scores[b, token_id] += lambda * log_prob`) rather than probability space. This preserves the numerical range of Whisper's logits and avoids saturating the softmax, which would collapse the beam to a single token. The `lambda = 0.5` interpolation weight was chosen empirically as the value that improves technical-word recall without reducing general speech WER.

Q2 — Language Identification: Non-Obvious Design Choice in Feature Pre-Extraction

Choice: Cache Wav2Vec2 embeddings to `.npz` before MLP training, rather than integrating the backbone into the training loop.

The straightforward approach is an end-to-end model where the transformer backbone and the classification head share a training graph. However, since the backbone's weights are **frozen** (no gradient propagation through Wav2Vec2), re-running the forward pass during every training iteration is purely redundant computation. On a CPU, a single Wav2Vec2 forward pass over a 1-second chunk takes approximately 300ms. With ~1,500 chunks and 200 training epochs, the naïve approach requires 1,500 × 200 = 300,000 forward passes — approximately 25 hours.

By extracting features once and caching them to `models/lid_features.npz` (~8.5 MB), training reduces to 200 iterations over pre-computed 768-dimensional vectors, completing in ~2 minutes. The `sklearn.neural_network.MLPClassifier` is used instead of PyTorch for the cached training step, as it is faster for small tabular data with no GPU. The learned weight matrices are then manually transferred to the `MultiHeadLID` PyTorch module by mapping `clf.coefs_` and `clf.intercepts_` to the corresponding `nn.Linear` layers, preserving downstream evaluate_all.py compatibility.

This pattern — **freeze → cache → tabular ML → port weights** — is a principled approach for self-supervised backbone + lightweight head architectures on resource-constrained hardware.

Q3 — DTW Prosody Warping: Non-Obvious Design Choice in Alignment Space

Choice: Compute DTW alignment on z-normalised energy sequences rather than directly on F0 contours.

F0 contours have a critical sparsity problem: they are defined only on voiced frames and are zero (undefined) during unvoiced consonants, fricatives, and pauses. In the Hinglish lecture, approximately 35% of frames are unvoiced. Attempting DTW directly on sparse F0 sequences produces a degenerate alignment that compresses or expands unvoiced regions erratically, causing prosodic artifacts in the warped output.

Energy (RMS, in dB) is dense and positive across all frames — voiced and unvoiced alike — and strongly correlates with the prosodic rhythm felt by a listener. By aligning on z-normalised energy:

```
E_norm[i] = (E[i] - mean(E)) / (std(E) + 1e-8)
```

the DTW path `π*` reliably captures rhythmic correspondences between source and target. The resulting monotone index path is then **transferred** to F0: `F0_warp[i_k] = F0_target[j_k]`, applying the prosodic rhythm in the energy domain to the pitch contour.

A second normalization step (z-score rescaling of voiced F0 frames, Eq. 10 in the paper) ensures the warped pitch occupies the source speaker's fundamental frequency range rather than the target TTS model's range. This prevents the synthesised Urdu from having the pitch characteristics of the MMS-TTS default voice, and instead preserves the source speaker's perceived vocal identity — which is the core objective of zero-shot voice cloning.

Q4 — Anti-Spoofing: Non-Obvious Design Choice in Feature Pooling

Choice: Concatenate frame-level mean and standard deviation of LFCC features (mean-std pooling) rather than using raw frame sequences with a recurrent classifier.

The straightforward approach for utterance-level anti-spoofing is to feed the full LFCC sequence `L ∈ R^(T×60)` into an LSTM or self-attention model. However, with only 31 training utterances (16 bona-fide + 15 spoof, each yielding 3-second segments), training a sequence model reliably is infeasible — recurrent models require at minimum hundreds of examples to avoid severe overfitting.

Mean-std pooling collapses the time axis into a 120-dimensional fixed-length representation:

```
ϕ(L) = [mean(L, axis=0) || std(L, axis=0)] ∈ R^120
```

This captures the **distributional statistics** of the LFCC trajectory, which differ systematically between natural and synthesized speech: TTS systems produce smoother, more predictable spectral trajectories (lower std) while natural speech has higher temporal variability. The 120-dim vector is then classified by a simple two-layer MLP, which trains stably to EER = 0.00% in 20 epochs on the 31-utterance dataset.

The insight is that anti-spoofing does not require modelling the exact temporal sequence — it requires distinguishing *statistical texture* of the spectrum. Mean-std pooling is the minimum sufficient statistic for this task when data is scarce.

All four design choices prioritise correctness and efficiency on CPU-constrained hardware, which reflects real-world deployment constraints for embedded speech understanding systems.